Documentation
for
IntelliLib(Library Management System)

April, 2026

Prepared By

Group Name: IntelliLib

Member Names:

1. Himani (2354058)
2. Aditya Kumar Jha (2354057)
3. Indranil Kundu (2354055)

Table of Contents

1 Introduction

1.1 Purpose

*The purpose of the Library Management System is to provide a **centralized, efficient, and automated platform** for managing all library operations, including book management, user management, issue and return transactions, and fine handling.*

*The system is designed to replace traditional manual processes with a **digital solution** that improves accuracy, reduces administrative workload, and enhances user experience for both librarians and members.*

It enables librarians to manage book records, track multiple copies, monitor issued and returned books, enforce issue limits, and handle overdue fines effectively. At the same time, it allows members to search for books, view availability, request books, track issued items, and manage fines and renewals.

*The system also ensures **data integrity, security, and transparency** by maintaining proper records of all transactions and activities through audit logs.*

Overall, the goal of the project is to:

1.1.1 *Improve **efficiency and speed** of library operations*
1.1.2 *Reduce **manual errors and paperwork***
1.1.3 *Provide **real-time information** about books and transactions*
1.1.4 *Ensure **fair usage through rules like issue limits and fines***
1.1.5 *Enhance **user convenience and accessibility***

1.2 Document Conventions

*This Software Requirements Specification (SRS) document follows the **IEEE standard format** for documenting software requirements. The following conventions are used throughout the document to ensure clarity and consistency:*

1.2.1 ***Standard Terminology:** Common software engineering terms are used consistently to describe system components, processes, and requirements.*
1.2.2 ***Requirement Identification:** Each functional requirement is uniquely identified using a structured format (e.g., REQ-4.1-1) for easy reference and traceability.*
1.2.3 ***Use of UML Diagrams:***
***Use Case Diagrams** are used to represent system functionalities and interactions between users (Member and Librarian) and the system.*
***Class Diagrams** represent the structural design of the system, including classes, attributes, methods, and relationships.*
***Sequence Diagrams** illustrate the interaction flow between system components over time.*
***Activity Diagrams** describe workflow and process logic.*
1.2.4 ***Data Modeling Notation:***
***Entity-Relationship Diagrams (ERD)** are represented using standard notation (entities, attributes, relationships, and cardinality).*

1.2.5 ***Data Flow Representation:***

Data Flow Diagrams (DFD) *use standard symbols such as processes (circles), data stores (open-ended rectangles), external entities (rectangles), and data flows (arrows).*

1.2.6 ***Formatting Conventions:***

Headings and subheadings are numbered hierarchically.
Important terms and features are highlighted using bold text where necessary.
Lists and bullet points are used for clarity and readability.

1.3 Project Scope

The Library Management System is a web-based software solution designed to ***automate and manage the core operations of a library*** *within an institution such as a college, university, or organization. The system provides a centralized platform for managing books, users, transactions, and administrative activities efficiently.*

The primary purpose of the system is to replace manual record-keeping with a ***digital, structured, and reliable system*** *that improves operational efficiency, reduces errors, and enhances accessibility for both librarians and members.*

Scope of the System

The system includes the following functionalities:

- *Management of* ***book records****, including adding, updating, deleting, and categorizing books.*
- *Handling* ***multiple copies of books*** *and tracking their real-time availability.*
- *Supporting* ***user account management*** *for members (students/staff) and librarians.*
- *Enabling* ***book issue and return operations*** *with proper validation and tracking.*
- *Enforcing* ***issue limits per user*** *to ensure fair usage of resources.*
- *Providing a* ***fine management system*** *to calculate and track penalties for overdue books.*
- *Allowing members to* ***search books, view availability, and access their borrowing history****.*
- *Supporting* ***book requests and renewal (extension) requests****, subject to librarian approval.*
- *Maintaining* ***audit logs and reports*** *for monitoring system activities and transactions.*

System Boundaries

The system is limited to managing library-related operations within a specific organization and does not include:

- *Physical inventory tracking beyond book records*
- *Integration with external library networks (unless added in future versions)*
- *Advanced digital content management (e.g., e-books or online reading platforms)*

Target Users

- ***Members:*** *Students or staff who use the system to search, borrow, and manage books.*
- ***Librarians:*** *Administrators who manage books, users, transactions, fines, and reports.*

Benefits of the System

- *Improves efficiency and reduces manual workload*
- *Minimizes errors in record-keeping*
- *Provides real-time information about books and transactions*
- *Enhances user convenience through easy search and access*
- *Ensures fair usage through automated rules (limits, fines, approvals)*

Future Enhancements

The system may be extended in future versions to include:

Online fine payment integration
Email/SMS notifications for due dates and reminders
Integration with digital libraries and e-book systems
Barcode/QR code-based book tracking
Mobile application support

1.4 References

he following references were used in the preparation of this Software Requirements Specification (SRS) document:

1.4.1 *IEEE*
IEEE Recommended Practice for Software Requirements Specifications (IEEE 830 / IEEE 29148 Standard)

1.4.2 *draw.io (or any diagramming tool you used such as Lucidchart)*
Used for creating UML diagrams including ER diagrams, DFDs, and other system models

1.4.3 *MySQL / MongoDB*
Used as a reference for database design and implementation concepts

1.4.4 *Visual Studio Code (or your development environment)*
Used for system development and code structuring

1.4.5 *Standard textbooks and online resources on **Software Engineering and System Design***

2 System Description

*The Library Management System is a web-based application designed to **automate and streamline the operations of a library**. It provides a centralized platform where librarians can manage books, users, and transactions, while members can access library services such as searching, borrowing, and tracking books.*

*The system improves efficiency by replacing traditional manual processes with a **digital, structured, and real-time system**, ensuring better accuracy, faster operations, and improved user experience.*

2.1 Existing System

In most traditional libraries, operations are managed manually or using basic record-keeping methods such as registers or spreadsheets. This approach has several limitations:

- *Manual entry of book and user records is **time-consuming and error-prone***
- *Difficulty in tracking **book availability and multiple copies***
- *Lack of proper tracking for **issued and returned books***
- *No automated system for **fine calculation and due date management***
- *Inefficient handling of **book requests and renewals***
- *Limited reporting and lack of **audit trails***
- *High chances of **data inconsistency and mismanagement***

2.2 Proposed System

*The proposed Library Management System introduces a **fully automated and centralized solution** to overcome the limitations of the existing system.*

Key improvements include:

- *Digital management of **books, users, and transactions***
- *Real-time tracking of **book availability and multiple copies***
- *Automated **book issuing and returning process***
- *Integrated **late fine calculation and restriction system***
- *Support for **book renewal requests and approvals***
- *Efficient **search functionality for books and categories***
- *Maintenance of **audit logs for all activities***
- *Generation of **reports for monitoring and decision-making***
- *Implementation of **role-based access control** for security*

2.3 Organizational Context

*The system is designed to operate within an institution such as a **college, university, or organization** where:*

- ***Librarians** act as administrators and manage all operations*
- ***Members (students/staff)** use the system to access library services*

The system supports a hierarchical structure where:

- *Librarians have full control over data and operations*
- *Members have limited access based on permissions*

2.4 System Overview

The system consists of the following major components:

- ***User Management Module:** Handles registration, login, and user roles*
- ***Book Management Module:** Manages books, categories, and copies*
- ***Transaction Module:** Handles issue, return, and renewal of books*
- ***Fine Management Module:** Calculates and tracks overdue fines*
- ***Search Module:** Enables users to find books quickly*
- ***Reporting and Logging Module:** Maintains reports and audit logs*

2.5 System Goals

The main goals of the system are:

*To **automate library operations** and reduce manual effort*
*To provide **accurate and real-time information***
*To ensure **fair usage through rules (limits, fines, approvals)***
*To improve **user experience and accessibility***
*To maintain **secure and reliable records of all activities***

3 Functional Requirements

This section contains system requirements followed by various requirement models which can be used for detailed design. In addition to the following sections, we have also added process flow diagrams, data flow diagrams, flowcharts and decision tables.

3.1 System Features

System Feature 1: User Management

The system shall allow users (members and librarians) to register, log in, and manage their accounts.

System Feature 2: Book Management

The system shall allow librarians to add, update, delete, and categorize books and manage multiple copies.

System Feature 3: Book Search and View

The system shall allow members to search for books, view categories, and check availability.

System Feature 4: Issue Book

The system shall allow librarians to issue books to members after validating availability and issue limits.

System Feature 5: Return Book

The system shall allow librarians to accept returned books and update records

accordingly.

System Feature 6: Fine Management

The system shall automatically calculate fines for overdue books and maintain fine records.

System Feature 7: Renewal Management

The system shall allow members to request renewal of books, subject to librarian approval.

System Feature 8: Book Request

The system shall allow members to request new books not available in the library.

System Feature 9: Reports and Logs

The system shall generate reports and maintain audit logs for all activities.

System Feature 10: Access Control

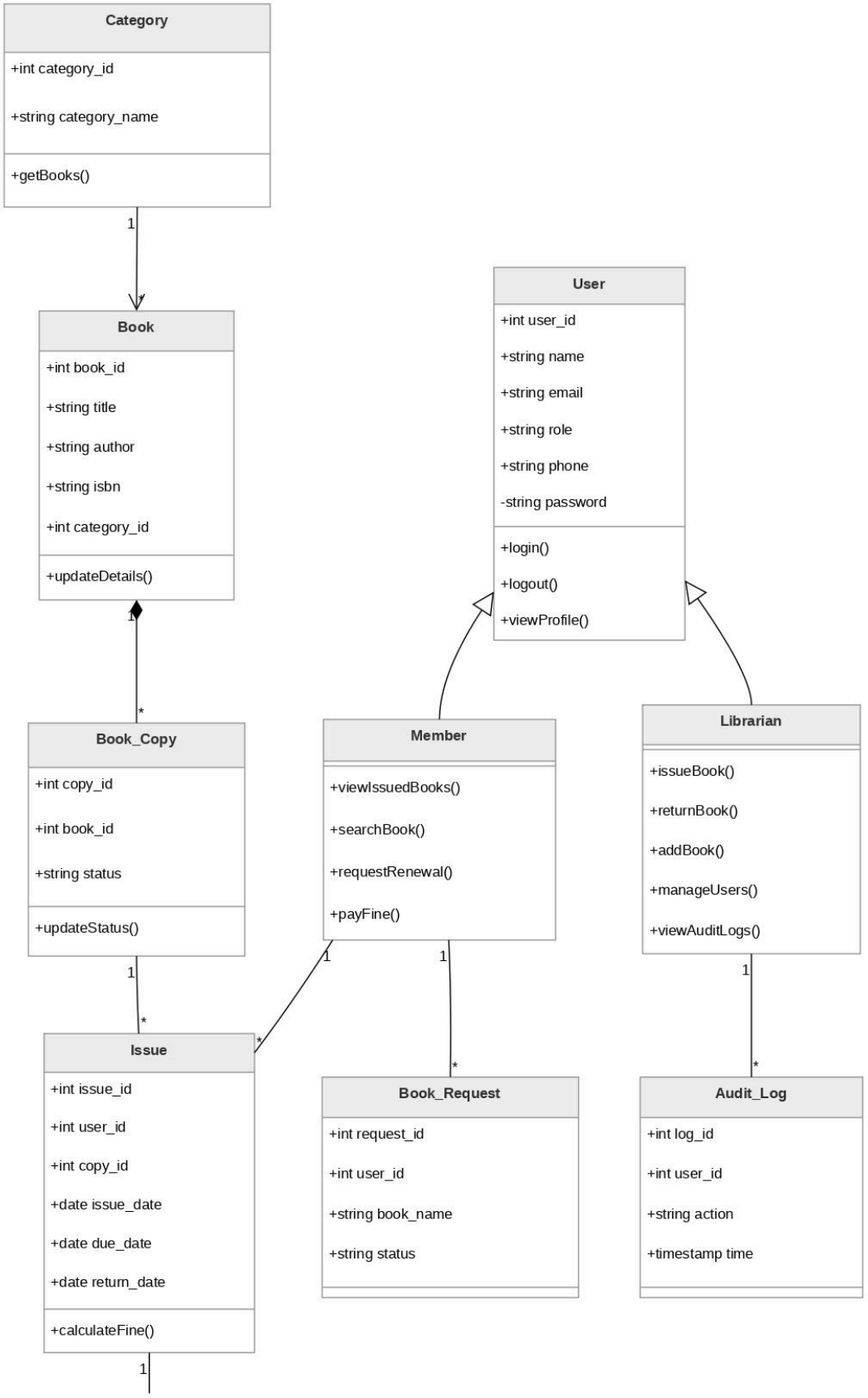
The system shall enforce role-based access for members and librarians.

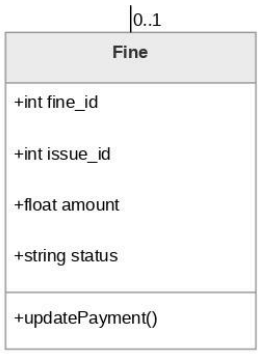
3.2 UML Diagrams

3.2.1 Class Diagrams

Description:

The class diagram represents the structure of the system including classes such as **User, Member, Librarian, Book, BookCopy, Issue, Fine, Request, and AuditLog**, along with their relationships and methods.


Category
+int category_id
+string category_name
+getBooks()
1
*
Book
+int book_id
+string title
+string author
+string isbn
+int category_id
+updateDetails()
1
*
Book_Copy
+int copy_id
+int book_id
+string status
+updateStatus()
1
*
User
+int user_id
+string name
+string email
+string role
+string phone
-string password
+login()
+logout()
+viewProfile()
Member
+viewIssuedBooks()
+searchBook()
+requestRenewal()
+payFine()
Librarian
+issueBook()
+returnBook()
+addBook()
+manageUsers()
+viewAuditLogs()
1
1
1
1
*
*
*
*
Issue
+int issue_id
+int user_id
+int copy_id
+date issue_date
+date due_date
+date return_date
+calculateFine()
1
Book_Request
+int request_id
+int user_id
+string book_name
+string status
Audit_Log
+int log_id
+int user_id
+string action
+timestamp time

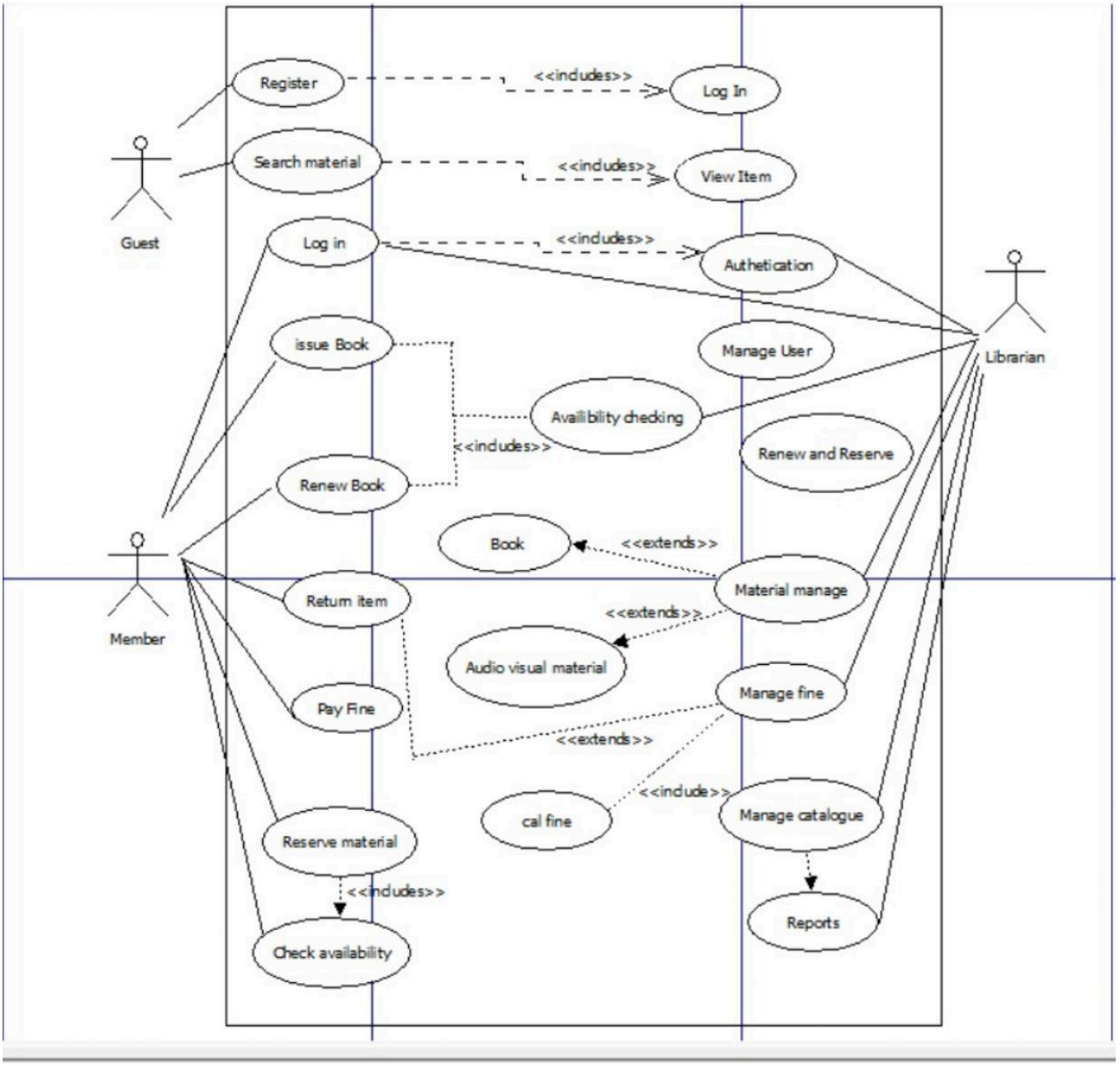

0..1
Fine
+int fine_id
+int issue_id
+float amount
+string status
+updatePayment()

3.2.2 Use Case Diagrams

<One or more diagrams depicting how various actors interact with the software system.>

<This provides a detailed description of the use case. Usually it is captured in the following table format. Add more rows or removing rows depending on your specific requirement.>

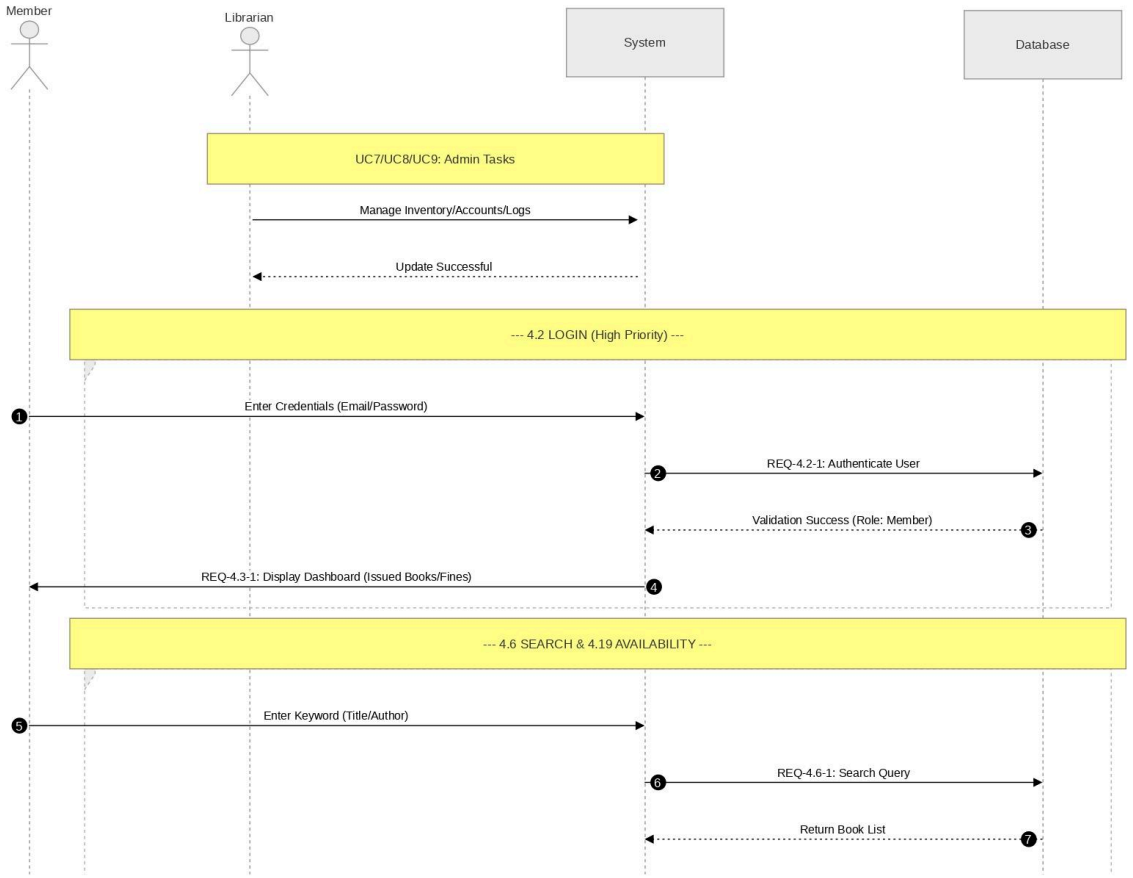
ID	*UC01*
Description	*Issue Book*
Actors	*Librarian*
Preconditions	User must be registered and book must be available
Basic Steps	*1. Librarian selects issue book 2. Enter user and book details 3. System checks availability and limit 4. Book is issued*
Alternate Steps	If book not available → queue
Exceptions	*System failure during transaction*
Business validations/Rules	*Issue limit must not be exceeded*
Postconditions	Book is marked as issued

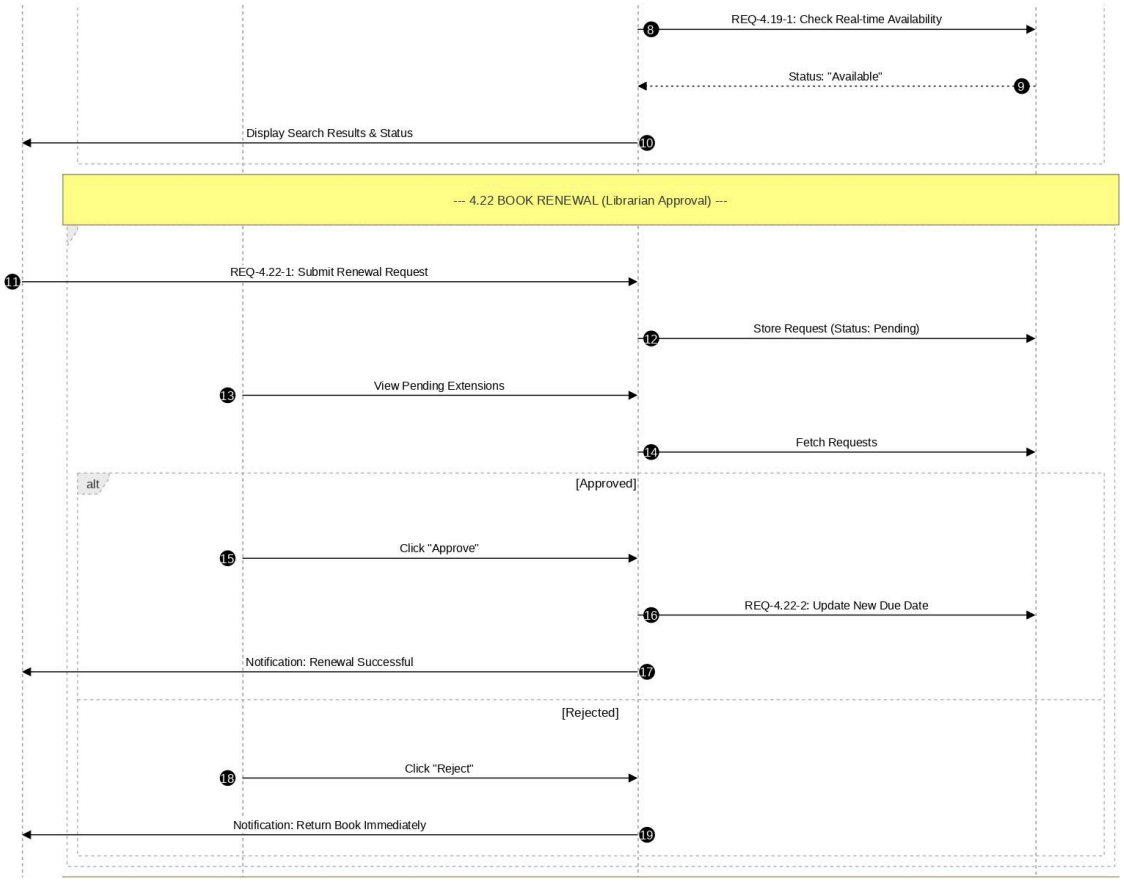

Register
<<includes>>
Log In
Search material
<<includes>>
View Item
Guest
Log in
<<includes>>
Authetication
Librarian
issue Book
Manage User
Availibility checking
<<includes>>
Renew and Reserve
Renew Book
Book
<<extends>>
Material manage
Member
Return item
<<extends>>
Audio visual material
Manage fine
Pay Fine
<<extends>>
<<include>>
cal fine
Manage catalogue
Reserve material
<<includes>>
Reports
Check availability

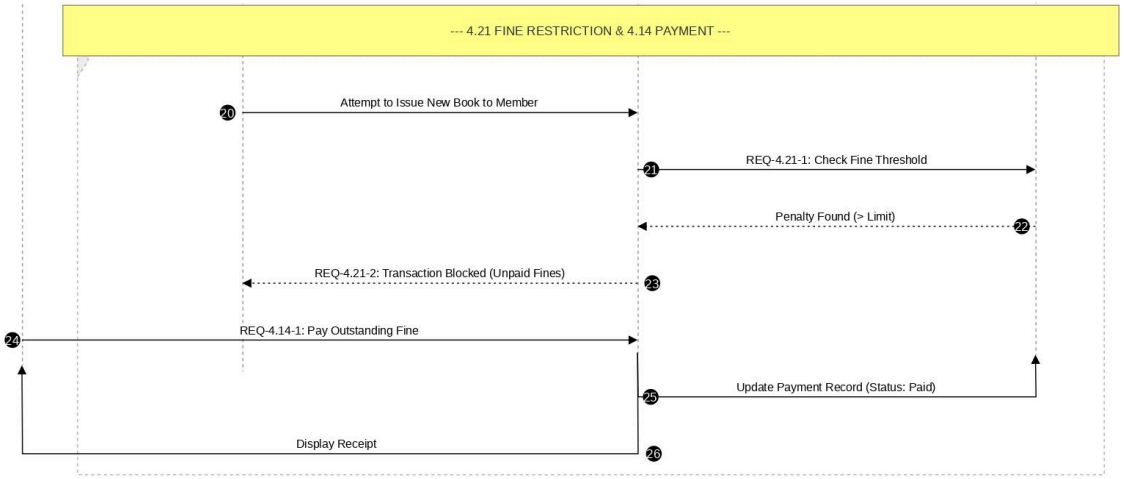
3.2.3 Sequence Diagram

Description:

Shows interaction between **Librarian → System → Database** and **Member → System or vice versa** during operations like issuing and returning books.



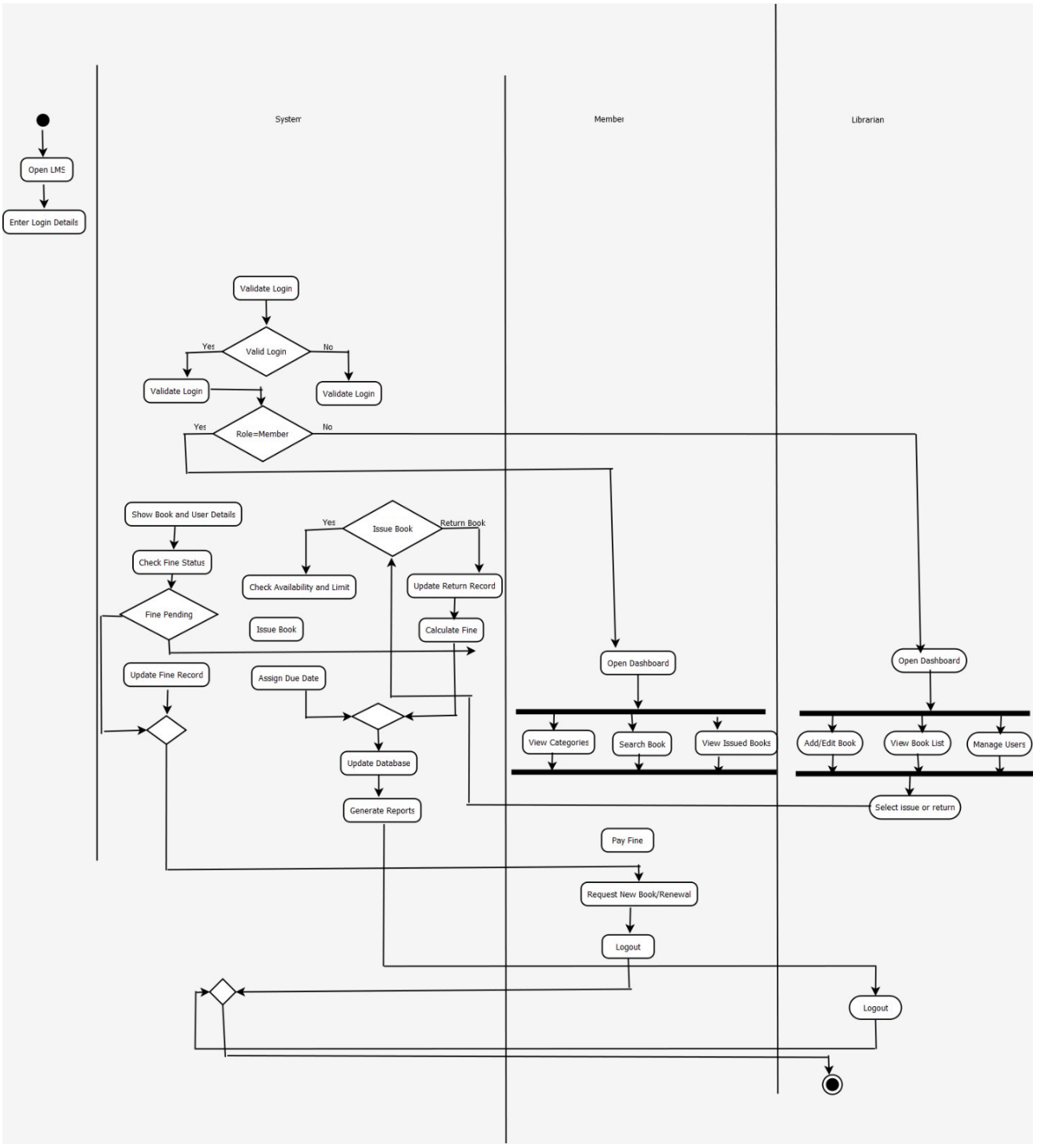

REQ-4.19-1: Check Real-time Availability
Status: "Available"
Display Search Results & Status
--- 4.22 BOOK RENEWAL (Librarian Approval) ---
REQ-4.22-1: Submit Renewal Request
Store Request (Status: Pending)
View Pending Extensions
Fetch Requests
alt
[Approved]
Click "Approve"
REQ-4.22-2: Update New Due Date
Notification: Renewal Successful
[Rejected]
Click "Reject"
Notification: Return Book Immediately


--- 4.21 FINE RESTRICTION & 4.14 PAYMENT ---
Attempt to Issue New Book to Member
REQ-4.21-1: Check Fine Threshold
Penalty Found (> Limit)
REQ-4.21-2: Transaction Blocked (Unpaid Fines)
REQ-4.14-1: Pay Outstanding Fine
Update Payment Record (Status: Paid)
Display Receipt

3.2.4 Activity Diagram

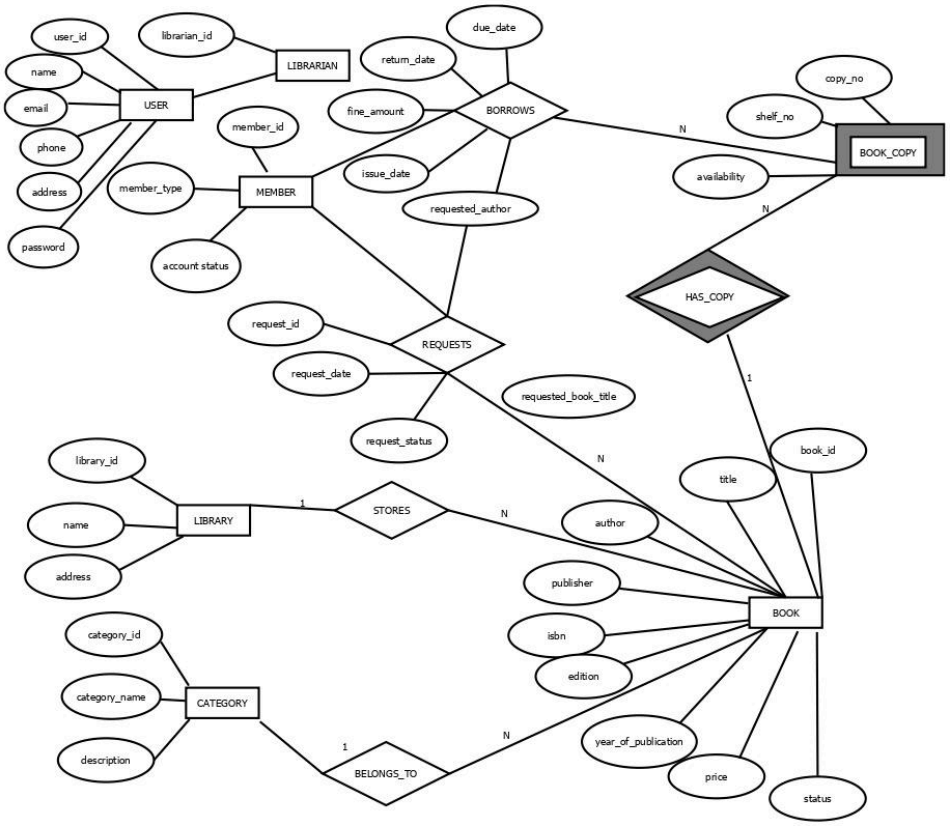
Description:

Represents workflows such as book issue, return, fine calculation, and renewal process.



3.3 Entity Relationship Diagrams

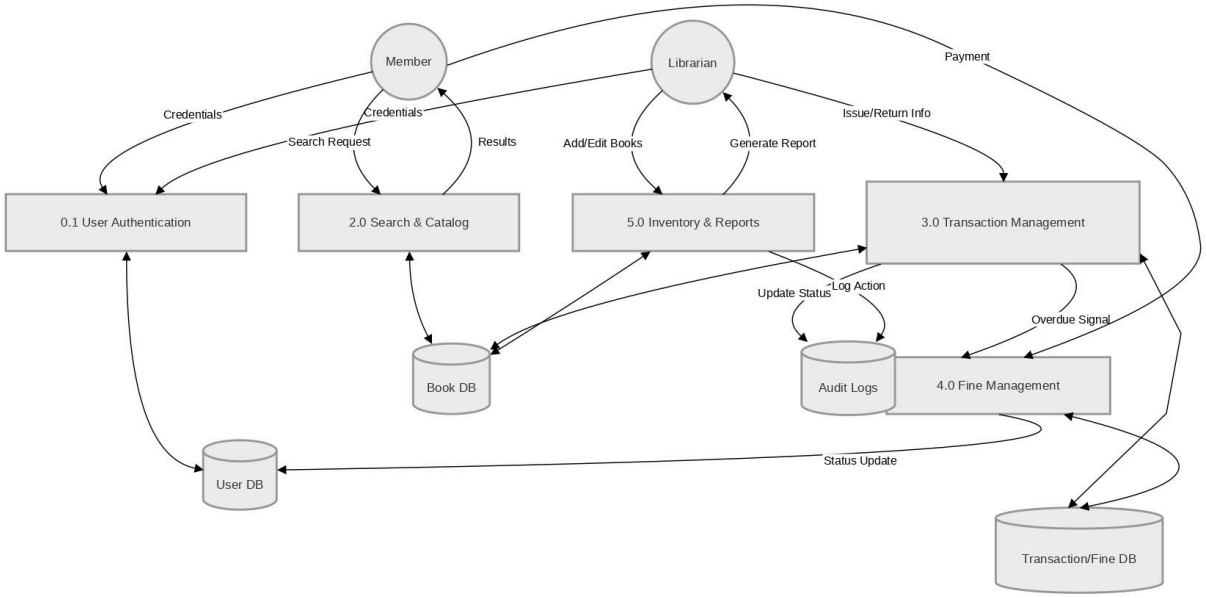
*Description:Shows entities such as **User, Book, BookCopy, Issue, Fine, Request, AuditLog** and their relationships.*


user_id
librarian_id
LIBRARIAN
due_date
return_date
name
email
USER
fine_amount
BORROWS
copy_no
shelf_no
BOOK_COPY
N
phone
member_id
address
member_type
MEMBER
issue_date
availability
requested_author
N
password
account status
HAS_COPY
request_id
REQUESTS
request_date
1
requested_book_title
request_status
book_id
N
library_id
title
1
STORES
N
LIBRARY
name
author
address
publisher
BOOK
category_id
isbn
edition
category_name
CATEGORY
1
N
year_of_publication
description
BELONGS_TO
price
status

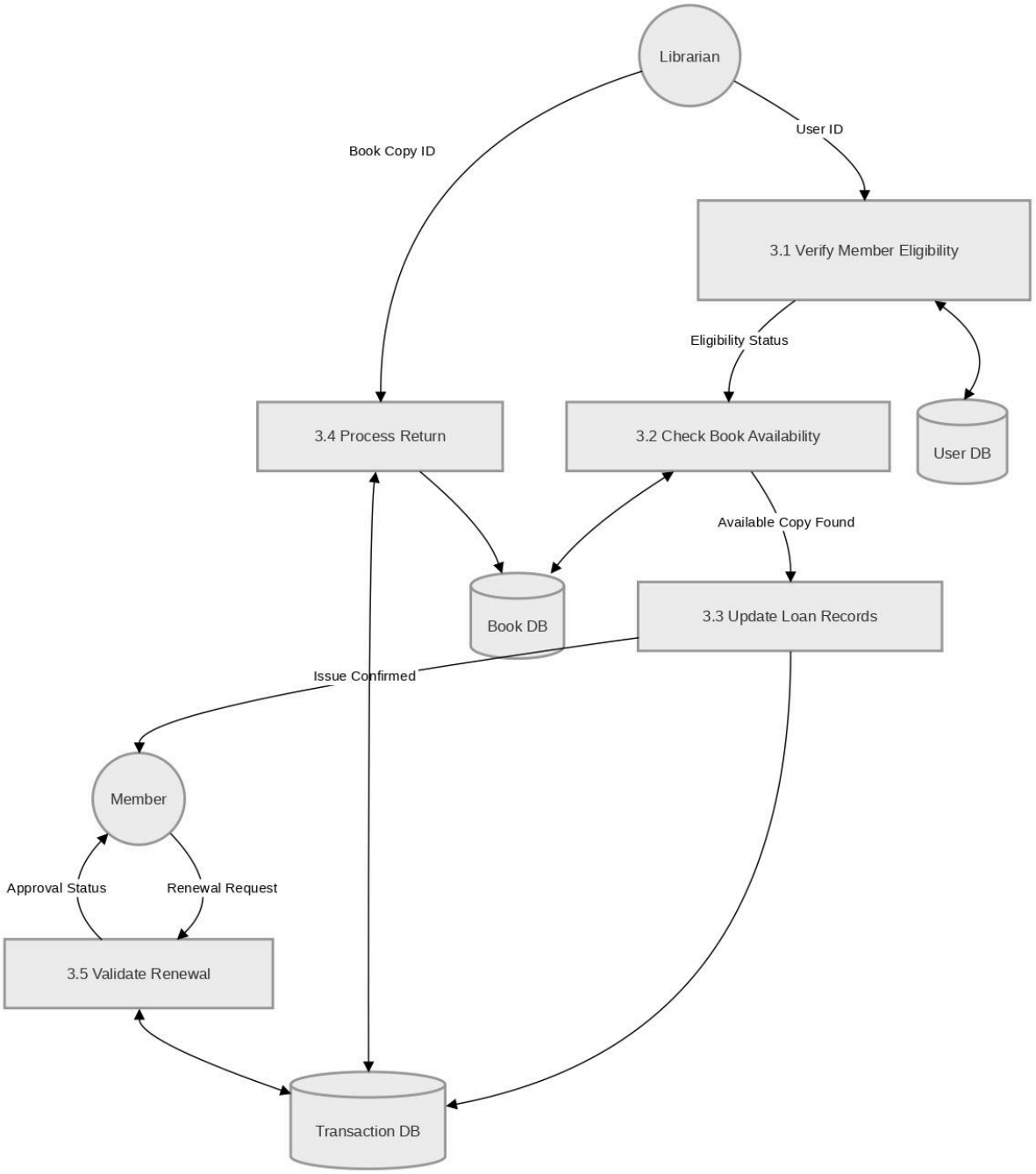
3.4Data Flow Diagram

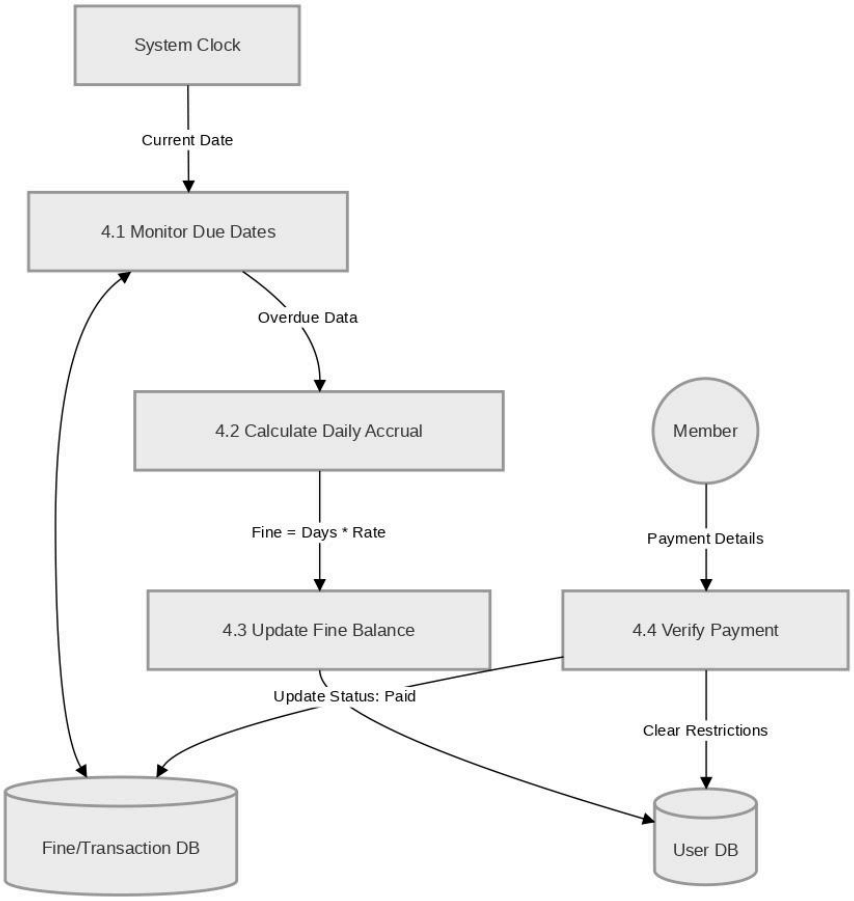
3.4.1 Context Level/ 0 Level DFD

3.4.2 First Level DFD



3.4.3 Second Level DFD




System Clock
Current Date
4.1 Monitor Due Dates
Overdue Data
4.2 Calculate Daily Accrual
Member
Fine = Days * Rate
Payment Details
4.3 Update Fine Balance
4.4 Verify Payment
Update Status: Paid
Clear Restrictions
Fine/Transaction DB
User DB

3.4 System Design

3.4.1 Database Schema Design

Schema Link

3.4.2 Project Directory Structure

```
intellilib/                          # Project root
├── README.md                        # Primary project documentation
├── package.json                     # Scripts and npm dependencies
├── Dockerfile.worker                # Notification worker container image
├── docker-compose.worker.yml        # Worker docker-compose service definition
├── docs/                            # Project and database docs
│   ├── reservation-system.md        # Reservation behavior notes
│   ├── sqlFile.txt                  # Historical SQL evolution log
│   ├── sql.dump.sql                 # One-shot canonical schema setup
│   └── sql.schema.md                # Human-readable schema + relationships
├── public/                          # Static assets served directly
│   ├── images/                      # Image assets grouped by feature
│   │   ├── demo/                    # README and demo screenshots
│   │   ├── hero/                    # Hero section model art
│   │   ├── heroSlider/              # Home slider book covers
│   │   ├── testimonials/            # Testimonial profile images
│   │   ├── contact/                 # 3D/contact section assets
│   │   └── giphy/                   # Animated GIF assets
├── scripts/                         # Operational scripts (seed/workers)
├── tests/                           # Test suites
└── src/                             # Main application source
    ├── app/                         # App Router pages, layouts, route handlers
    │   ├── page.tsx                 # Public landing page
    │   ├── layout.tsx               # Root layout and providers
    │   ├── globals.css              # Global app styling
    │   ├── not-found.tsx            # Global 404 page
    │   ├── opengraph-image.tsx      # Dynamic OG social image
    │   ├── twitter-image.tsx        # Dynamic Twitter card image
    │   ├── robots.ts                # Robots directives
    │   ├── sitemap.ts               # Sitemap generator
```

```
│   ├── favicon.ico                    # Browser favicon
│   ├── icon.svg                       # App icon asset
│   ├── discover/page.tsx              # Discover books page
│   ├── library/page.tsx               # Library overview page
│   ├── login/page.tsx                 # Login page
│   ├── signup/page.tsx                # Signup page
│   ├── search/page.tsx                # Public search page
│   ├── search/[id]/page.tsx           # Book details page
│   ├── cookies/page.tsx               # Cookie policy page
│   ├── privacy/page.tsx               # Privacy policy page
│   ├── terms/page.tsx                 # Terms page
│   ├── delete-account/page.tsx        # Account deletion policy page
│   ├── dashboard/                     # Role-aware dashboard route group
│   │   ├── layout.tsx                 # Shared dashboard layout
│   │   ├── page.tsx                   # Dashboard root redirect/entry
│   │   ├── user/layout.tsx            # User dashboard layout
│   │   ├── user/page.tsx              # User dashboard home
│   │   ├── user/[section]/page.tsx    # Dynamic user dashboard sections
│   │   ├── librarian/layout.tsx       # Librarian dashboard layout
│   │   ├── librarian/page.tsx         # Librarian dashboard home
│   │   ├── librarian/[section]/page.tsx  # Dynamic librarian dashboard sections
│   │   ├── admin/layout.tsx           # Admin dashboard layout
│   │   ├── admin/page.tsx             # Admin dashboard home
│   │   └── admin/[section]/page.tsx   # Dynamic admin dashboard sections
│   └── api/                           # Backend API route handlers
│       ├── debug/members/route.ts     # Debug-only members endpoint
│       ├── library/search/route.ts    # Public catalog search API
│       ├── library/books/[id]/route.ts  # Book details API
│       ├── library/issue/route.ts     # Member issue request API
│       ├── library/returns/route.ts   # Member return request API
│       ├── library/reservations/route.ts  # Reservation CRUD API
│       ├── library/bookmarks/route.ts  # Bookmark CRUD API
│       ├── library/assistant/route.ts  # User AI assistant API
│       ├── library/queue/process/route.ts  # Manual queue processing API
│       ├── library/queue/process/cron/route.ts# Cron-protected queue processing API
│       ├── library/queue/worker-health/route.ts# Worker health status API
```

```
│   ├── librarian/catalog/route.ts        # Librarian catalog list/create API
│   ├── librarian/catalog/[id]/route.ts   # Catalog update/delete API
│   ├── librarian/catalog/[id]/copies/route.ts # Copy-level update API
│   ├── librarian/issue/route.ts          # Librarian issue desk API
│   ├── librarian/return/route.ts         # Librarian return desk API
│   ├── librarian/requests/route.ts       # Librarian requests processing API
│   ├── librarian/assistant/route.ts      # Librarian AI assistant API
│   ├── librarian/assistant/action/route.ts# Assistant action execution API
│   ├── members/toggle/route.ts           # Member suspend/reactivate API
│   ├── razorpay/create-order/route.ts    # Razorpay order creation API
│   └── razorpay/verify/route.ts          # Razorpay signature verification API
├── components/                           # Reusable UI and domain components
│   ├── HomePageComponents/               # Marketing/landing section components
│   │   ├── AiSmartDiscovery.tsx          # AI capabilities section
│   │   ├── DashboardPreview.tsx          # Dashboard preview section
│   │   ├── HeroSlider.tsx                # Hero carousel
│   │   ├── Testimonials.tsx              # Testimonials section
│   │   ├── UseCase.tsx                   # Use case cards section
│   │   └── contact/                      # Contact section 3D scene components
│   │       ├── Computer.tsx              # 3D computer model component
│   │       ├── Contact.tsx               # Contact section container
│   │       └── ContactExperience.tsx     # 3D scene experience setup
│   ├── auth/                             # Authentication form components
│   │   ├── AnimationPanel.tsx            # Visual side panel for auth pages
│   │   ├── LoginForm.tsx                 # Email/password login form
│   │   ├── OtpForm.tsx                   # OTP verification form
│   │   ├── RoleGuard.tsx                 # Role-based UI render guard
│   │   ├── SignUpForm.tsx                # User registration form
│   │   ├── SocialButtons.tsx             # OAuth/social login buttons
│   │   └── lottie/                       # Lottie animation JSON files
│   │       ├── animation-1.json          # Auth animation asset 1
│   │       ├── animation-2.json          # Auth animation asset 2
│   │       └── animation-3.json          # Auth animation asset 3
│   ├── common/                           # Shared shell and utility components
│   │   ├── Dropdown.tsx                  # Generic dropdown component
│   │   ├── Footer.tsx                    # Site footer
```

```
│   │   ├── Navbar.tsx                  # Site navigation bar
│   │   ├── PaginationControls.tsx      # Paginated list controls
│   │   └── TitleHeader.tsx             # Reusable section/page title header
│   ├── dashboard/                      # Dashboard shared and role components
│   │   ├── DashboardRoleLayout.tsx     # Role-aware dashboard layout wrapper
│   │   ├── DashboardSectionPlaceholder.tsx  # Placeholder for missing sections
│   │   ├── DashboardShell.tsx          # Shared dashboard shell
│   │   ├── DashboardStatCard.tsx       # Reusable stats card
│   │   ├── admin/                      # Admin dashboard components
│   │   │   ├── AIIntelligencePanel.tsx     # AI/system intelligence insights panel
│   │   │   ├── AlertsAttentionPanel.tsx    # Alerts and attention-required panel
│   │   │   ├── CompactMetricCard.tsx       # Compact metric card primitive
│   │   │   ├── FinancialSnapshotPanel.tsx  # Finance KPI snapshot panel
│   │   │   ├── InventoryHealthPanel.tsx    # Inventory health panel
│   │   │   ├── LibraryActivitySnapshot.tsx  # Library activity summary panel
│   │   │   ├── PanelCard.tsx           # Panel card wrapper
│   │   │   ├── RecentActivityFeed.tsx      # Recent admin activity feed
│   │   │   ├── TrendSparkline.tsx      # Sparkline mini chart component
│   │   │   ├── UsageTrendPanel.tsx     # Usage trends visualization panel
│   │   │   ├── UserInsightsPanel.tsx   # User analytics/insights panel
│   │   │   └── data.ts                 # Admin dashboard data helpers
│   │   ├── librarian/                  # Librarian dashboard components
│   │   │   ├── ActionRequiredPanel.tsx     # Pending tasks/actions panel
│   │   │   ├── FinancialSnapshotPanel.tsx  # Librarian finance overview panel
│   │   │   ├── InventorySnapshotPanel.tsx  # Inventory overview panel
│   │   │   ├── LibrarianPanelCard.tsx      # Librarian panel card primitive
│   │   │   ├── LibrarianStatsRow.tsx       # Stats row strip
│   │   │   ├── LiveActivityFeedPanel.tsx   # Real-time activity feed panel
│   │   │   ├── MemberActivityInsightsPanel.tsx# Member behavior insights panel
│   │   │   ├── MiniTrendsPanel.tsx         # Mini trends visualization panel
│   │   │   ├── NotificationsPreviewPanel.tsx  # Notifications quick preview panel
│   │   │   ├── QuickActionsPanel.tsx       # Shortcut action buttons panel
│   │   │   ├── SystemEfficiencyPanel.tsx   # Operations efficiency panel
│   │   │   ├── data.ts                 # Librarian dashboard data helpers
│   │   │   ├── useLibrarianDashboardData.ts  # Librarian data-fetch/composition hook
│   │   │   ├── analytics/              # Analytics subpages/panels
```

```
│   │   │   ├── AnalyticsPage.tsx          # Analytics full page container
│   │   │   ├── AnalyticsPanel.tsx         # Analytics summary panel
│   │   │   ├── LiveActivityPanel.tsx      # Live activity panel server wrapper
│   │   │   └── LiveActivityPanelClient.tsx# Live activity client renderer
│   │   ├── assistant/                     # Librarian assistant UI components
│   │   │   └── LibrarianAssistantSection.tsx # Assistant chat/actions section
│   │   ├── audit/                         # Audit logs UI
│   │   │   ├── AuditLogEntry.tsx          # Single audit entry renderer
│   │   │   ├── AuditLogSection.tsx        # Audit section container
│   │   │   └── AuditLogSectionClient.tsx  # Client audit interactions
│   │   ├── catalog/                       # Catalog management UI
│   │   │   ├── CatalogSection.tsx         # Catalog management section
│   │   │   └── LibrarianBookCard.tsx      # Catalog book card item
│   │   ├── circulation/                   # Issue/return desk UI
│   │   │   ├── CirculationDesk.tsx        # Circulation desk actions component
│   │   │   ├── CirculationPage.tsx        # Circulation page container
│   │   │   ├── CirculationPanel.tsx       # Circulation metrics/tasks panel
│   │   │   └── CirculationStatsRow.tsx    # Circulation stats row
│   │   ├── members/                       # Member management UI
│   │   │   ├── MembersPage.tsx            # Members page container
│   │   │   ├── MembersPanel.tsx           # Members summary panel
│   │   │   ├── MembersStatsRow.tsx        # Members stats row
│   │   │   └── MembersTable.tsx           # Members data table
│   │   └── requests/                      # Request handling UI
│   │       ├── RequestsPage.tsx           # Requests page container
│   │       ├── RequestsPanel.tsx          # Requests summary panel
│   │       └── RequestsTable.tsx          # Requests table/actions
│   └── user/                              # User dashboard components
│       ├── AIUsageInsightsPanel.tsx       # AI usage insights panel
│       ├── ActionRequiredCard.tsx         # Required actions callout card
│       ├── ContinueReadingPanel.tsx       # Continue reading panel
│       ├── PersonalizedInsightsPanel.tsx  # Personalized insights panel
│       ├── QuickActionsPanel.tsx          # User quick actions panel
│       ├── ReadingActivityStats.tsx       # Reading stats row/charts
│       ├── SmartNotificationsPanel.tsx    # Smart notification panel
│       ├── UserPanelCard.tsx              # User panel card primitive
```

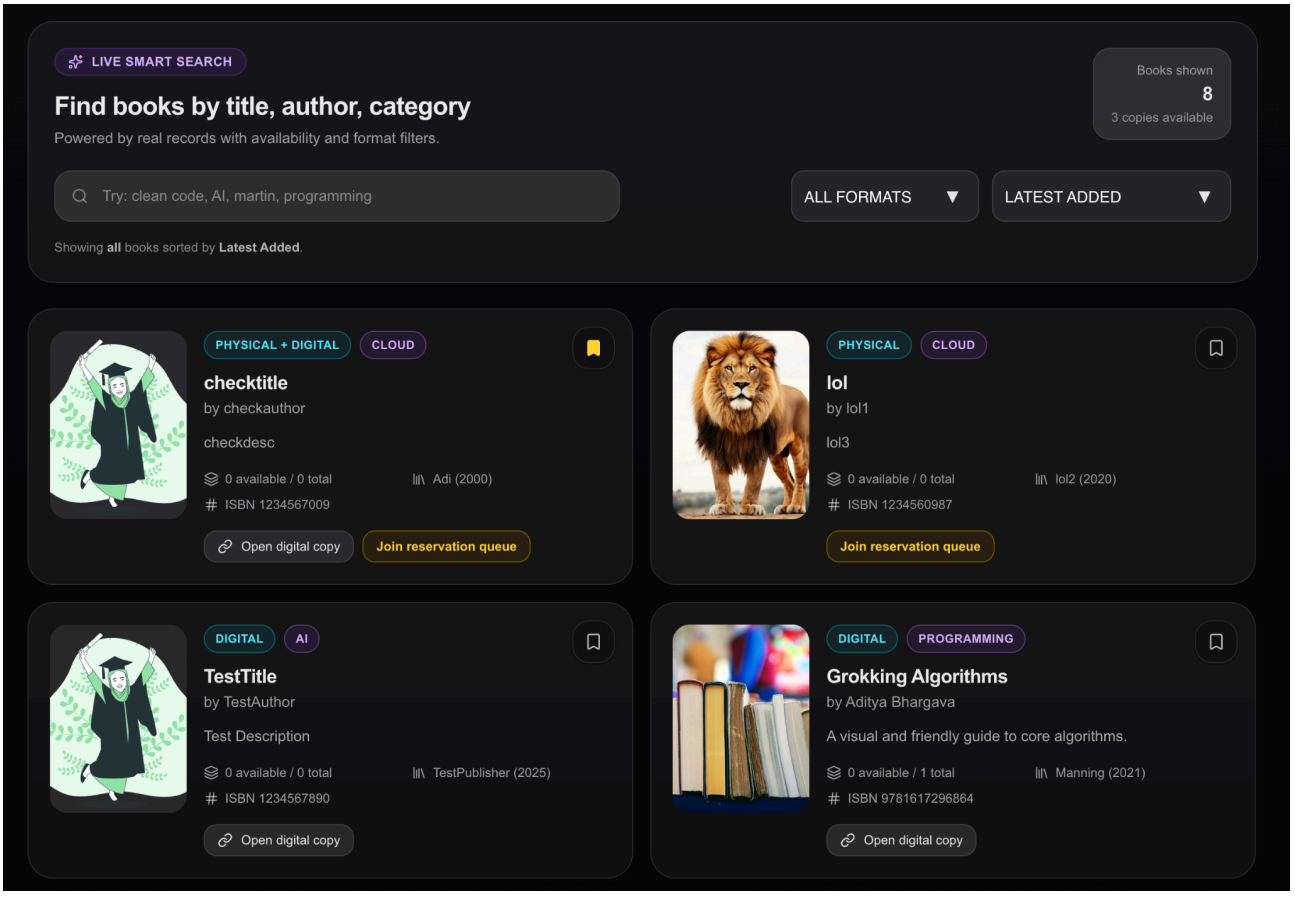
```
│   │   ├── UserRecentActivityFeed.tsx     # User activity feed panel
│   │   ├── UserStatsRow.tsx               # User stats row
│   │   ├── data.ts                        # User dashboard data helpers
│   │   ├── assistant/                     # User assistant UI
│   │   │   └── UserAssistantSection.tsx   # User assistant section
│   │   ├── bookmarks/                     # Bookmarks feature module
│   │   │   ├── BookmarkToggleButton.tsx   # Toggle bookmark button
│   │   │   ├── UserBookmarksSection.tsx   # User bookmarks section
│   │   │   ├── bookmark-utils.ts          # Bookmark helper utilities
│   │   │   ├── types.ts                   # Bookmark feature types
│   │   │   └── useUserBookmarkIds.ts      # Bookmark IDs hook
│   │   ├── fines/                         # Fine/payment feature UI
│   │   │   └── UserFinesSection.tsx       # User fines and payment section
│   │   ├── history/                       # Reading/transaction history
│   │   │   └── UserHistorySection.tsx     # User history section
│   │   ├── my-books/                      # Current issued books feature
│   │   │   ├── MyBookIssueCard.tsx        # Issued book card
│   │   │   ├── MyBooksStatsRow.tsx        # Issued books stats row
│   │   │   ├── UserMyBooksSection.tsx     # My books section
│   │   │   ├── my-books-utils.ts          # My books helper utilities
│   │   │   └── types.ts                   # My books feature types
│   │   ├── notifications/                 # User notifications module
│   │   │   └── NotificationsSection.tsx   # Notifications list section
│   │   ├── reservations/                  # Reservation queue module
│   │   │   ├── ReservationQueueCard.tsx   # Reservation queue card
│   │   │   ├── UserReservationsSection.tsx# Reservations section
│   │   │   ├── reservation-utils.ts       # Reservation helper utilities
│   │   │   └── types.ts                   # Reservation feature types
│   │   └── search/                        # User smart search module
│   │       ├── BookSearchResultCard.tsx   # Search result item card
│   │       ├── UserSmartSearchSection.tsx # Smart search section
│   │       ├── search-utils.ts            # Search helper utilities
│   │       └── types.ts                   # Search feature types
│   ├── search/                            # Public search UI components
│   │   ├── PublicBookCard.tsx             # Public book card
│   │   ├── PublicBookDetailPage.tsx       # Public book detail view
```

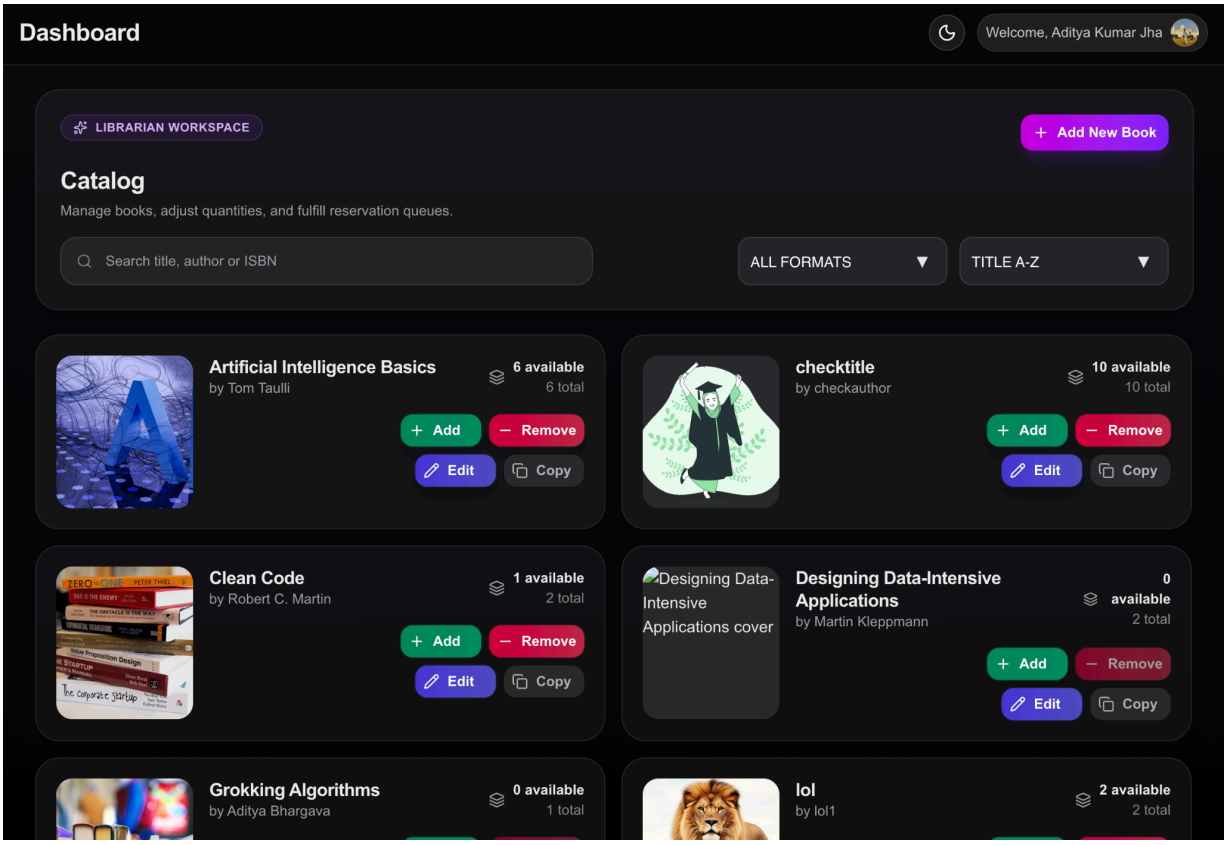
```
│   │   ├── PublicBookmarkButton.tsx        # Public bookmark action
│   │   ├── PublicSearchClient.tsx          # Client search orchestration
│   │   ├── PublicSearchFilters.tsx         # Public filter sidebar/toolbar
│   │   ├── PublicSearchPage.tsx            # Public search page container
│   │   ├── search-utils.ts                 # Public search helper utilities
│   │   └── types.ts                        # Public search types
│   └── ui/                                 # Base UI primitives/providers
│       ├── Button.tsx                      # Primary button component
│       ├── badge-button.tsx                # Badge-like button primitive
│       ├── badge.tsx                       # Badge primitive
│       ├── hover-expand.tsx                # Hover-expand interaction primitive
│       ├── minimal-card.tsx                # Minimal card primitive
│       ├── query-provider.tsx              # React Query provider wrapper
│       ├── theme-animations.ts             # Theme animation definitions
│       ├── theme-provider.tsx              # Theme context/provider
│       ├── theme-toggle-button.tsx         # Theme toggle UI button
│       └── themeButton.tsx                 # Alternate theme button component
├── lib/                                    # Shared logic/utilities and server services
│   ├── authStore.ts                        # Client auth store (Zustand)
│   ├── authUser.ts                         # Auth user normalization helpers
│   ├── dashboardNav.ts                     # Role-based dashboard nav config
│   ├── errorMessage.ts                     # Error message extraction/format helpers
│   ├── seo.ts                              # SEO metadata helpers
│   ├── supabaseClient.ts                   # Browser Supabase client
│   ├── supabaseServerClient.ts             # Server/service-role Supabase client
│   ├── utils.ts                            # Generic utility helpers
│   └── server/                             # Server-only business services
│       ├── adminAnalytics.ts               # Admin dashboard aggregation logic
│       ├── apiAuth.ts                      # Bearer token auth helper for APIs
│       ├── auditLogs.ts                    # Audit log write helpers
│       ├── catalogService.ts               # Catalog fetch/update service layer
│       ├── imagekit.ts                     # ImageKit integration helpers
│       ├── librarianCirculation.ts         # Librarian issue/return business rules
│       ├── librarianRequests.ts            # Librarian request processing rules
│       ├── libraryNotifications.ts         # Notification creation/queueing service
│       ├── members.ts                      # Member profile/status management service
```

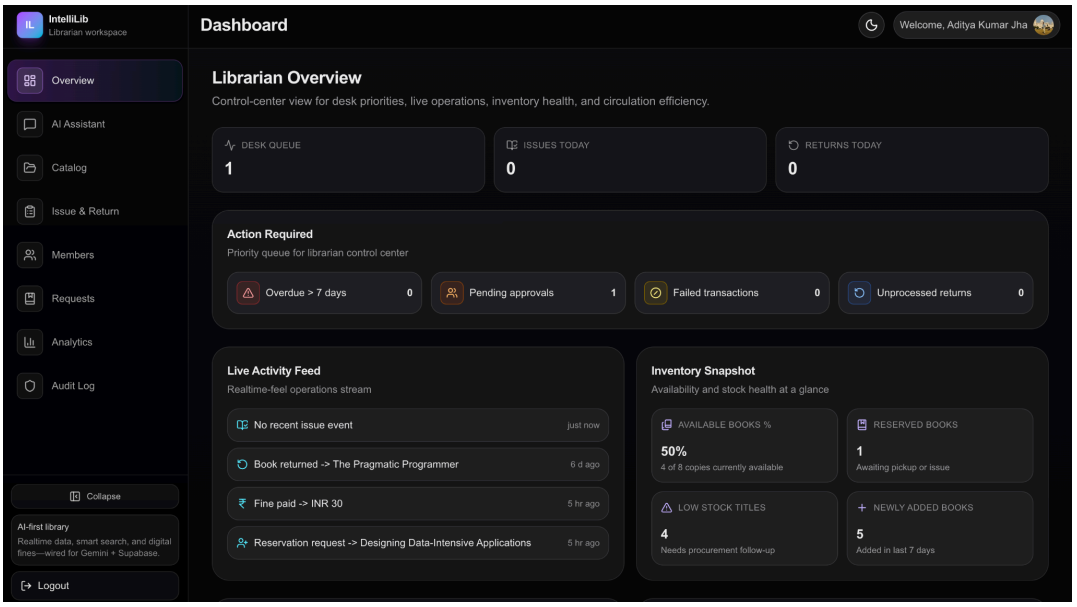
```
│   ├── queueProcessor.ts          # Reservation/notification queue processor
│   ├── reservationService.ts      # Reservation queue and policy logic
│   └── suspensionGuard.ts         # Access guard for suspended users
├── pages/                         # Legacy Pages Router compatibility layer
│   ├── _app.tsx                   # Legacy app wrapper
│   ├── HomePage.tsx               # Legacy home wrapper/export
│   ├── AuthPage.tsx               # Legacy auth page wrapper/export
│   ├── AdminDashboardPage.tsx     # Legacy admin dashboard wrapper/export
│   ├── LibrarianDashboardPage.tsx # Legacy librarian dashboard wrapper/export
│   └── UserDashboardPage.tsx      # Legacy user dashboard wrapper/export
└── styles/                        # Additional style modules
    ├── style.css                  # Shared style utilities
    └── Dropdown.module.css        # Dropdown component styles
```

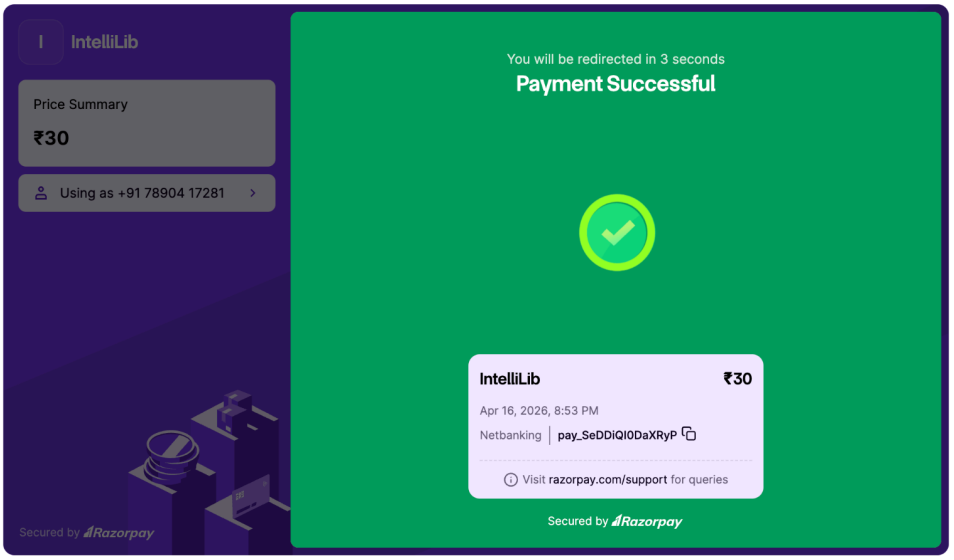
.....

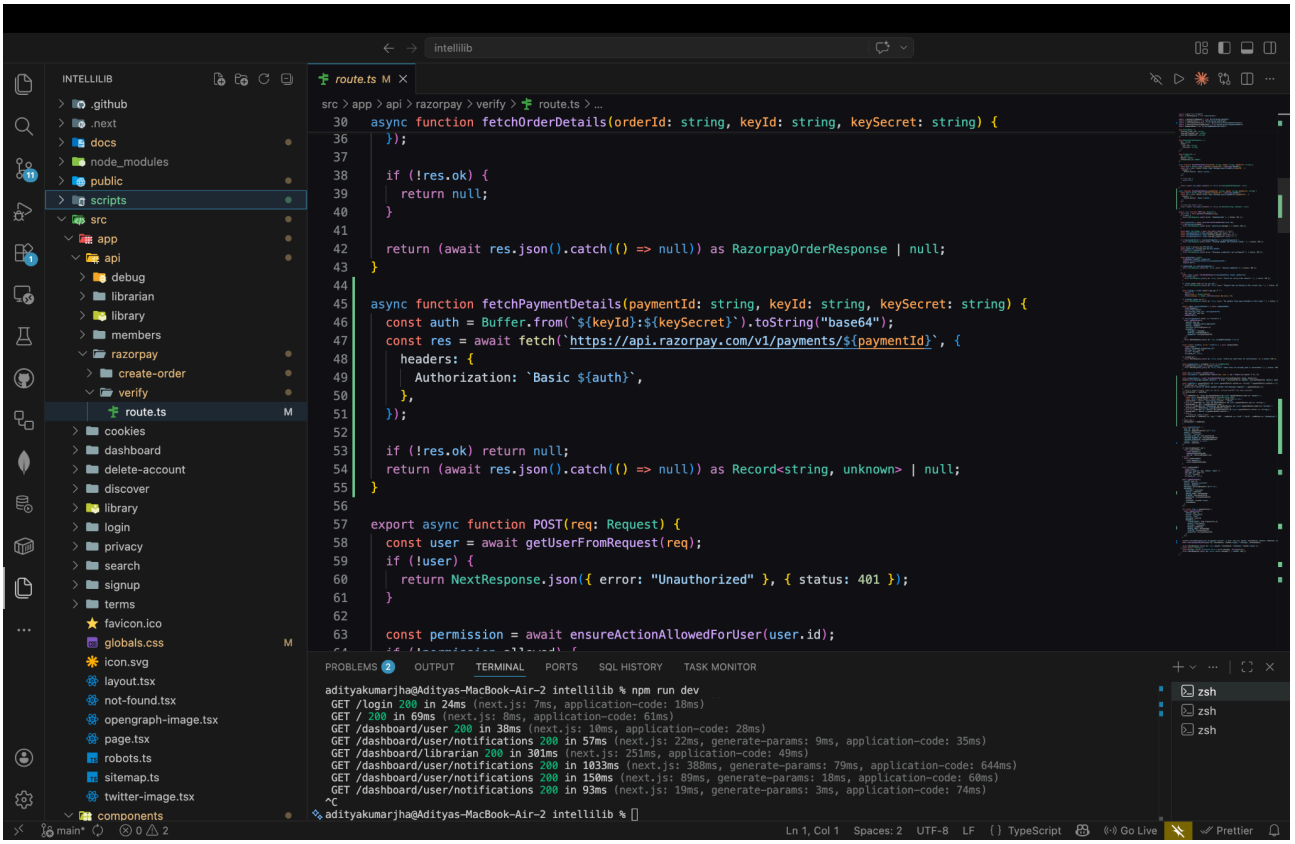
3.5 Screen Shots from Projects

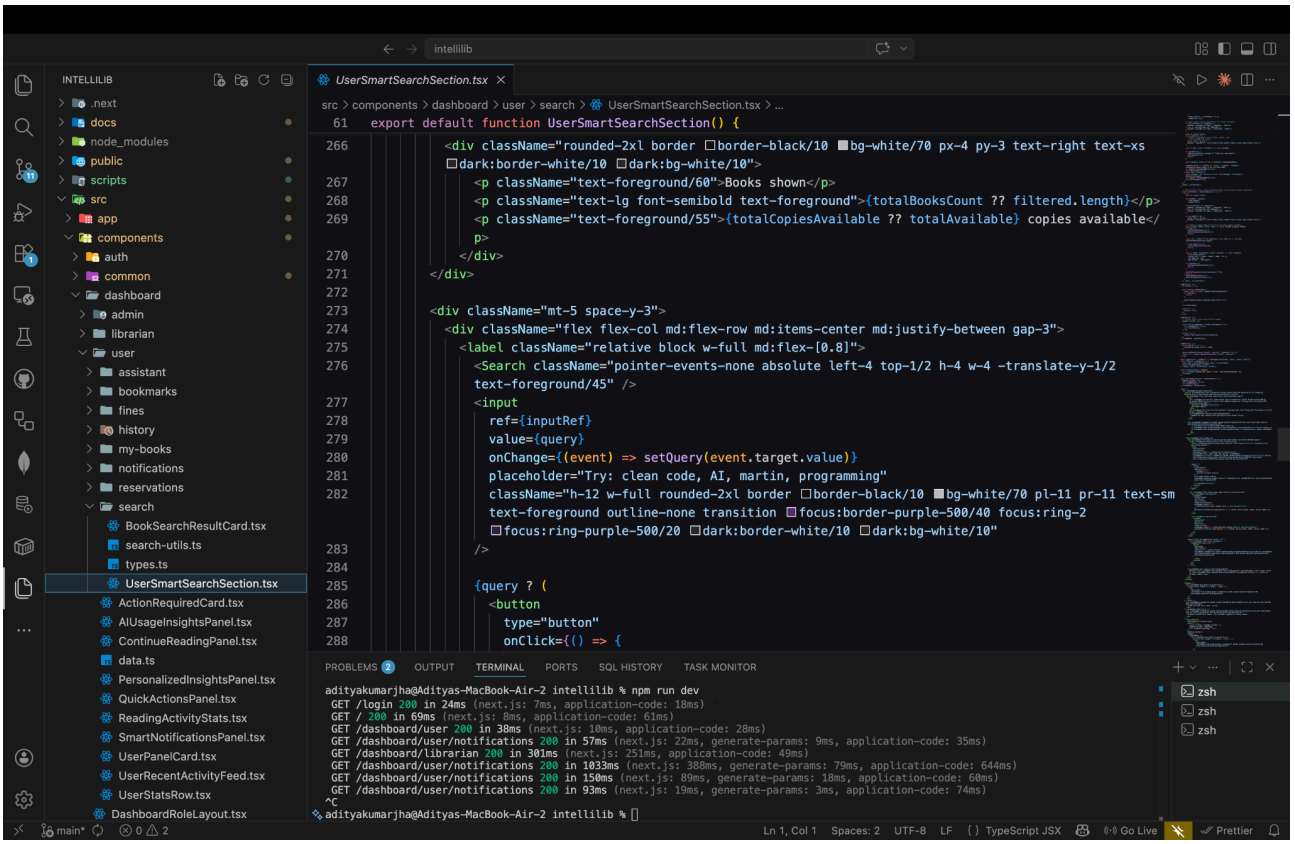



Dashboard
Welcome, Aditya Kumar Jha
LIBRARIAN WORKSPACE
Add New Book
Catalog
Manage books, adjust quantities, and fulfill reservation queues.
Search title, author or ISBN
ALL FORMATS
TITLE A-Z
Artificial Intelligence Basics
by Tom Taulli
6 available
6 total
Add
Remove
Edit
Copy
checktitle
by checkauthor
10 available
10 total
Add
Remove
Edit
Copy
Clean Code
by Robert C. Martin
1 available
2 total
Add
Remove
Edit
Copy
Designing Data-Intensive Applications cover
Designing Data-Intensive Applications
by Martin Kleppmann
0 available
2 total
Add
Remove
Edit
Copy
Grokking Algorithms
by Aditya Bhargava
0 available
1 total
lol
by lol1
2 available
2 total


IntelliLib
Librarian workspace
Dashboard
Welcome, Aditya Kumar Jha
Overview
AI Assistant
Catalog
Issue & Return
Members
Requests
Analytics
Audit Log
Collapse
AI-first library
Realtime data, smart search, and digital fines—wired for Gemini + Supabase.
Logout
Librarian Overview
Control-center view for desk priorities, live operations, inventory health, and circulation efficiency.
DESK QUEUE
1
ISSUES TODAY
0
RETURNS TODAY
0
Action Required
Priority queue for librarian control center
Overdue > 7 days
0
Pending approvals
1
Failed transactions
0
Unprocessed returns
0
Live Activity Feed
Realtime-feel operations stream
No recent issue event
just now
Book returned -> The Pragmatic Programmer
6 d ago
Fine paid -> INR 30
5 hr ago
Reservation request -> Designing Data-Intensive Applications
5 hr ago
Inventory Snapshot
Availability and stock health at a glance
AVAILABLE BOOKS %
50%
4 of 8 copies currently available
RESERVED BOOKS
1
Awaiting pickup or issue
LOW STOCK TITLES
4
Needs procurement follow-up
NEWLY ADDED BOOKS
5
Added in last 7 days


IntelliLib
Price Summary
₹30
Using as +91 78904 17281
You will be redirected in 3 seconds
Payment Successful
IntelliLib
₹30
Apr 16, 2026, 8:53 PM
Netbanking
pay_SeDDiQI0DaXRyP
Visit razorpay.com/support for queries
Secured by Razorpay
Secured by Razorpay


route.ts
async function fetchOrderDetails(orderId: string, keyId: string, keySecret: string) {
async function fetchPaymentDetails(paymentId: string, keyId: string, keySecret: string) {
export async function POST(req: Request) {
adityakumarjha@Adityas-MacBook-Air-2 intellilib % npm run dev
GET /login 200 in 24ms
GET / 200 in 69ms
GET /dashboard/user 200 in 38ms
GET /dashboard/user/notifications 200 in 57ms
GET /dashboard/librarian 200 in 301ms
GET /dashboard/user/notifications 200 in 1033ms
GET /dashboard/user/notifications 200 in 150ms
GET /dashboard/user/notifications 200 in 93ms


INTELLILIB
UserSmartSearchSection.tsx
src > components > dashboard > user > search > UserSmartSearchSection.tsx > ...
export default function UserSmartSearchSection() {
PROBLEMS 2
OUTPUT
TERMINAL
PORTS
SQL HISTORY
TASK MONITOR
adityakumarjha@Adityas-MacBook-Air-2 intellilib % npm run dev
GET /login 200 in 24ms (next.js: 7ms, application-code: 18ms)
GET / 200 in 69ms (next.js: 8ms, application-code: 61ms)
GET /dashboard/user 200 in 38ms (next.js: 10ms, application-code: 28ms)
GET /dashboard/user/notifications 200 in 57ms (next.js: 22ms, generate-params: 9ms, application-code: 35ms)
GET /dashboard/librarian 200 in 301ms (next.js: 251ms, application-code: 49ms)
GET /dashboard/user/notifications 200 in 1033ms (next.js: 388ms, generate-params: 79ms, application-code: 644ms)
GET /dashboard/user/notifications 200 in 150ms (next.js: 89ms, generate-params: 18ms, application-code: 60ms)
GET /dashboard/user/notifications 200 in 93ms (next.js: 19ms, generate-params: 3ms, application-code: 74ms)
^C
adityakumarjha@Adityas-MacBook-Air-2 intellilib %
zsh
zsh
zsh